

A PROJECT REPORT ON

**Deep Reinforcement Learning for Supply Chain
Optimization: A DQN and LSTM-Based
Approach**

**SUBMITTED TO THE SAVITRIBAI PHULE UNIVERSITY, PUNE
IN THE PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE**

OF

BACHELOR OF ENGINEERING (Computer Engineering)

SUBMITTED BY

Mihir Deshpande	Exam No: B400050506
Vibhav Saharabudhe	Exam No: B400050574
Piyush Agarwal	Exam No: B400050552
Atharva Zodpe	Exam No: B400050641



**DEPARTMENT OF COMPUTER ENGINEERING
Pune Institute of Computer Technology
Dhankawadi, Pune - 411043**

**SAVITRIBAI PHULE PUNE UNIVERSITY
2024-2025**



CERTIFICATE

This is to certify that the project report entitled

**Deep Reinforcement Learning for Supply Chain Optimization:
A DQN and LSTM-Based Approach**

Submitted by

Mihir Deshpande	Exam No: B400050506
Vibhav Saharabudhe	Exam No: B400050574
Piyush Agarwal	Exam No: B400050552
Atharva Zodpe	Exam No: B400050641

is a bonafide student of this institute and the work has been carried out by him/her under the supervision of **Prof. M.S. Chavan** and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Prof. M. S. Chavan
Internal Guide
Dept. of Computer Engg.

Dr. G. V. Kale
Head
Dept. of Computer Engg.

Dr. S. T. Gandhe
Principal
Pune Institute of Computer Technology

Place:Pune
Date: 24/04/2025



Project Completion with ATX

Date: April 14, 2025

TO WHOMSOEVER IT MAY CONCERN

This is to certify that the following students:

- Mihir Deshpande
- Atharva Zodpe
- Vibhav Sahasrabudhe
- Piyush Agarwal

have successfully completed their Bachelor of Engineering (BE) project titled:

“Reinforcement Learning in Manufacturing Domain for Supply Chain Optimization” at ATX Labs under the guidance of Mr. Suraj Singh (Head of Engineering).

The project was carried out as part of their academic curriculum and demonstrated a commendable application of Reinforcement Learning techniques to address real-world challenges in the manufacturing and supply chain sector. Throughout the project, the students exhibited technical proficiency, innovative thinking, and a collaborative approach in solving complex problems.

We appreciate their dedication and wish them continued success in their future endeavours.

A handwritten signature in purple ink, appearing to read 'Suraj Singh', with a stylized flourish at the end. Below the signature, there is a small, faint red stamp or logo.

Thanks & Regards,
Suraj Singh,
Head of AI Engineering
ATX Labs

ACKNOWLEDGEMENT

*It gives us great pleasure in presenting the project report on ‘**Deep Reinforcement Learning for Supply Chain Optimization: A DQN and LSTM-Based Approach**’.*

*We would like to take this opportunity to thank **Prof. M. S. Chavan** giving me all the help and guidance we needed. We are really grateful to them for their kind support. Their valuable suggestions were very helpful.*

*We are also grateful to **Dr. G. V. Kale**, Head of Computer Engineering Department, PICT for his indispensable support, suggestions.*

*In the end our special thanks to **Mr. Suraj Singh**, ATX Labs and all faculty members for their whole hearted cooperation for completion of this report. We also thank our laboratory assistants for their valuable help in laboratory.*

Mihir Deshpande
Vibhav Sahasrabudhe
Piyush Agarwal
Atharva Zodpe
(B.E. Computer Engg.)

ABSTRACT

Effective inventory management plays a key role in improving supply chains by maintaining the right stock levels, reducing holding costs, and avoiding stockouts. Traditional systems often struggle to respond to real-time changes in demand and supply. In this project, we explore a Reinforcement Learning (RL)-based method for dynamic inventory optimization using Deep Q-Networks (DQN) along with Multi-Armed Bandit (MAB) strategies like Epsilon-Greedy, Upper Confidence Bound (UCB), KL-UCB, and Thompson Sampling.

The DQN agent learns the best way to manage inventory by interacting with a simulated environment and adjusting decisions based on changing demand patterns. We compare these techniques using performance indicators such as inventory cost, stockout rate, and overall supply chain efficiency. Our results show that while bandit-based methods offer good starting strategies, the DQN approach performs better in the long run, especially in handling uncertainty and making adaptive decisions. This demonstrates how deep reinforcement learning can help supply chains become more responsive, cost-effective, and resilient.

Contents

1	Introduction	1
1.1	Overview	2
1.2	Motivation	3
1.3	Problem Definition and Objectives	4
1.4	Project Scope & Limitations	6
1.5	Methodologies of Problem solving	8
2	Literature Survey	10
2.1	Literature Review	11
3	Software Requirements Specification	13
3.1	Assumptions and Dependencies	14
3.1.1	Assumptions	14
3.1.2	Dependencies	14
3.2	Functional Requirements	16
3.2.1	Data Management	16
3.2.2	Reinforcement Learning Model Development	16
3.2.3	Decision-Making Capabilities	16
3.2.4	Continuous Learning and Adaptation	16
3.2.5	User Interface	17
3.2.6	Reporting and Analytics	17
3.3	External Interface Requirements	18
3.3.1	User Interfaces	18
3.3.2	Software Interfaces	18
3.3.3	Hardware Interfaces	18
3.3.4	Communication Interfaces	19
3.4	Nonfunctional Requirements	20
3.4.1	Performance Requirements	20
3.4.2	Safety and Security Requirements	20
3.5	System Requirements	21
3.5.1	Hardware Requirements	21

3.5.2	Software Requirements	21
3.6	Analysis Models: SDLC Model to be applied	22
4	System Design	23
4.1	System Architecture	24
4.1.1	1. User Interface Layer	24
4.1.2	2. Application Logic	24
4.1.3	3. Simulation Environment	24
4.1.4	4. Data Management	24
4.1.5	5. Integration Layer	25
4.1.6	6. Infrastructure	25
4.2	Mathematical Model	26
4.2.1	Objective Function	26
4.2.2	Policy Optimization Methods	27
4.2.3	Constraints	27
4.3	Data Flow Diagrams	28
4.4	Entity Relationship Diagram	29
4.5	Use Case Diagram	30
4.6	Sequence Diagram	31
5	Project Plan	32
5.1	Project Estimate	33
5.1.1	Reconciled Estimates	33
5.1.2	Project Resources	33
5.2	Risk Management	35
5.2.1	Risk Identification	35
5.2.2	Risk Analysis	36
5.2.3	Overview of Risk Mitigation, Monitoring, and Man-	
	agement	37
5.3	Project Schedule	38
5.3.1	Project Task Set	38
5.3.2	Task Network	40
5.4	Team Organization	42
5.4.1	Team Members and Roles	42
5.4.2	Management, Reporting and Communication	43
6	Project Implementation	44
6.1	Overview of Project Modules	45
6.2	Tools and Technologies Used	46
6.3	Algorithms Used	47
6.3.1	Algorithm 1: Q-Learning	47

6.3.2	Algorithm 2: Deep Q-Network (DQN)	47
6.3.3	Algorithm 3: Proximal Policy Optimization (PPO)	48
6.3.4	Algorithm 4: Long Short-Term Memory (LSTM)	48
7	Software Testing	49
7.1	Type of Testing	50
7.1.1	Unit Testing	50
7.1.2	Integration Testing	50
7.1.3	System Testing	50
7.1.4	Performance Testing	50
7.1.5	User Acceptance Testing (UAT)	50
7.2	Test Cases and Results	51
8	Results	52
8.1	Outcomes	53
8.1.1	Successful Integration of Reinforcement Learning Model	53
8.1.2	Streamlit-Based Interactive User Interface	53
8.1.3	Improved Inventory Control and Scheduling Efficiency	53
8.1.4	Seamless Backend-Frontend Communication	54
8.1.5	Forecasting using LSTM for Demand Prediction	54
8.1.6	Stakeholder Validation and Feedback	54
8.1.7	Scalable and Modular Codebase	54
8.2	Screenshots	55
9	Conclusions	56
9.1	Conclusions	57
9.2	Future Work	58
9.3	Applications	58
10	Appendix	59
10.1	Appendix A: Problem Statement Feasibility Assessment	60
	Appendix A: Problem Statement Feasibility Assessment	60
10.2	Appendix B	62
10.2.1	Paper Publication	62
	Appendix D: Paper Publication	62
10.2.2	Certificates	63
	Appendix E: Certificates	63
10.3	Appendix C	64
11	References	65

List of Figures

3.1 Iterative Model for SDLC	22
4.1 System Architecture	25
4.2 Data Flow Diagram	28
4.3 ER Diagram	29
4.4 Use Case Diagram	30
4.5 Sequence Diagram	31
8.1 Graphical View	55
8.2 Tabular View	55
10.1 Certificates of Paper Publication	63
10.2 Plagiarism Check	64

CHAPTER 1

INTRODUCTION

1.1 Overview

Efficient inventory management stands as a cornerstone of effective supply chain operations, directly impacting profitability, customer satisfaction, and operational agility. Traditional inventory control systems often rely on static rule-based methods or heuristics, which struggle to adapt to the dynamic nature of real-world supply and demand patterns. These limitations become even more pronounced in high-variability environments where decisions must continuously adapt to fluctuating customer behavior, lead times, and holding costs.

In this project, we explore a novel approach to inventory optimization using Deep Reinforcement Learning (DRL), specifically combining Deep Q-Networks (DQN) with Long Short-Term Memory (LSTM) networks. Our methodology allows the model to not only learn optimal ordering policies from past experiences but also capture temporal patterns in demand and supply sequences—something conventional methods fail to address adequately. The integration of LSTM enhances the agent’s ability to model sequential dependencies, which is crucial for long-term reward optimization in supply chain scenarios.

The core objective of the project is to minimize total inventory-related costs—primarily stockouts and holding costs—while maximizing long-term rewards. To this end, we develop a custom Warehouse Environment using OpenAI Gym standards, wherein the agent learns to make inventory decisions through interactions and delayed feedback. Comparative analyses have been conducted between a baseline rule-based policy, a standard DQN agent, and our proposed DQN+LSTM hybrid model.

Our results demonstrate promising improvements. The DQN+LSTM model achieved an average reward of 1450, outperforming both the rule-based system (850) and standard DQN (1260). The stockout rate dropped to 3.2 percent, and holding costs were reduced by 31 percent, signifying a tangible advancement in inventory management strategies. These metrics affirm the effectiveness of deep learning-based reinforcement models in environments characterized by delayed and sparse rewards.

This work highlights how combining state-of-the-art reinforcement learning techniques with sequence modeling can lead to robust, self-adaptive inventory policies that improve operational efficiency and decision-making accuracy in supply chain systems.

1.2 Motivation

Inventory management is a critical yet complex function within supply chain operations. Organizations across industries struggle to strike a balance between understocking—which leads to lost sales and poor customer satisfaction—and overstocking, which incurs excessive holding costs and wastage. Traditional inventory policies such as Economic Order Quantity (EOQ), (s, S) models, or manually tuned heuristics lack the flexibility and adaptability required to thrive in highly dynamic market environments. These static methods typically assume demand distributions and lead times to be known and stationary, which is rarely the case in real-world scenarios.

Our motivation for this project arises from these limitations and the growing need for intelligent, data-driven decision-making in supply chain systems. The recent advancements in Reinforcement Learning (RL) offer an exciting opportunity to reimagine inventory optimization from a learning-based perspective, where agents can learn optimal ordering policies through trial-and-error interactions with an environment. However, standard RL methods often fall short when faced with non-Markovian dynamics—especially when historical context or temporal demand patterns significantly influence future states and rewards.

To bridge this gap, we explore the use of Deep Q-Networks (DQN) integrated with Long Short-Term Memory (LSTM) networks. This combination empowers the learning agent to consider both immediate states and sequential patterns in historical data, thus improving its ability to anticipate future demands and act accordingly. Furthermore, the inclusion of Multi-Armed Bandit (MAB) techniques for action exploration enhances the agent's ability to balance exploration and exploitation, especially during early training stages.

From a broader perspective, our project is motivated by the need to bring scalable and adaptive AI solutions to industries such as retail, manufacturing, pharmaceuticals, and logistics, where inventory inefficiencies can lead to millions in losses annually. By applying deep reinforcement learning, we aim to design an intelligent inventory control policy that continuously improves over time, reduces operational costs, and delivers robust performance even in uncertain and volatile environments.

This project not only serves as a proof of concept for applying DQN + LSTM in inventory control but also lays the groundwork for extending reinforcement learning techniques to other areas of supply chain and operations research.

1.3 Problem Definition and Objectives

Problem Definition:

Efficient inventory management is central to ensuring uninterrupted supply chain operations while minimizing overhead costs. Conventional rule-based or statistical methods struggle to adapt to environments where demand fluctuates unpredictably, lead times vary, and feedback delays exist. These challenges are further compounded in multi-product, multi-period settings where each decision can affect long-term profitability and service levels. The core problem addressed in this project is:

To develop an adaptive and intelligent inventory control mechanism that learns to place optimal replenishment orders over time in a stochastic, dynamic environment, with the goal of minimizing stockouts and holding costs while maximizing overall operational reward.

Unlike static optimization approaches, our focus lies in **model-free reinforcement learning**, where the environment's dynamics are not known a priori, and the agent learns optimal strategies through experience.

Objectives:

To tackle the stated problem, we define the following key objectives:

- **Design a Simulated Inventory Environment:** Construct a custom Gym-based simulation that closely mimics real-world inventory behavior with variable demand patterns, lead times, holding costs, and penalty for stockouts.
- **Apply Deep Q-Networks (DQN):** Implement a standard DQN algorithm as the baseline reinforcement learning agent, capable of approximating the Q-values for state-action pairs using a deep neural network.
- **Incorporate LSTM into the DQN Architecture:** Enhance the standard DQN model by integrating a Long Short-Term Memory (LSTM) layer to capture temporal dependencies and leverage historical trends in demand, enabling more informed ordering decisions.
- **Introduce Multi-Armed Bandit (MAB) for Exploration:** Utilize MAB-based strategies to improve exploration during early training stages and address the sample inefficiency of standard ϵ -greedy approaches.

- **Optimize Performance Metrics:** Evaluate the model using critical KPIs such as:
 - Average total reward per episode
 - Stockout rate
 - Holding cost reduction
 - Ordering policy efficiency over training
- **Compare Against Rule-Based Baseline:** Benchmark the proposed models (Standard DQN and DQN + LSTM) against traditional rule-based approaches to demonstrate their superiority in performance and adaptability.

These objectives align with the broader vision of enabling **autonomous decision-making in supply chains**, reducing manual intervention, and improving overall system resilience.

1.4 Project Scope & Limitations

Scope:

This project focuses on leveraging Deep Reinforcement Learning (DRL) techniques to improve decision-making in inventory management systems. Specifically, it implements and evaluates Deep Q-Networks (DQN) and an enhanced DQN model with Long Short-Term Memory (LSTM) layers to optimize stock replenishment strategies in a simulated warehouse environment.

The key areas covered under the project scope include:

- **Simulation Environment:** Design and implementation of a custom OpenAI Gym-compatible inventory management environment that simulates real-world conditions such as variable lead times, stochastic demand, and fluctuating holding and shortage costs.
- **Model Development:** Implementation of both standard DQN and DQN + LSTM architectures for learning optimal inventory policies. The LSTM integration enables the model to account for temporal dependencies in demand trends.
- **Training and Evaluation:** Models are trained over multiple episodes, and their performance is compared against a rule-based baseline using metrics such as average reward, stockout rate, and holding cost reduction.
- **Analysis and Visualization:** In-depth analysis of trends in rewards, penalties, and policy evolution is carried out to understand the learning dynamics and model efficiency.

Limitations:

Despite the promising results, the project has some limitations:

- **Simplified Environment:** The simulation environment, while representative, abstracts many complexities found in real-world multi-product, multi-node supply chains (e.g., supplier constraints, transportation delays, perishability).
- **Scalability:** The current models are optimized for a single-item inventory scenario. Scaling the architecture to manage multiple SKUs (Stock Keeping Units) would require further architectural tuning and computational resources.

- **Generalization:** The trained models are environment-specific. Applying them to different inventory scenarios would require retraining or transfer learning mechanisms.
- **Assumption of Stationarity:** The models assume that the demand distribution and system parameters remain stationary over time, which may not hold in dynamic real-world markets.

1.5 Methodologies of Problem solving

The problem of optimizing inventory management in the face of uncertain and dynamic demand patterns is approached using Deep Reinforcement Learning (DRL). The solution methodology comprises the following major phases:

1. **Environment Design:** A custom simulation environment is created, adhering to OpenAI Gym standards. This environment models a single-product inventory system, incorporating:

- Randomized daily demand with Gaussian noise to simulate variability.
- Fixed and stochastic lead times for incoming inventory.
- Cost functions for stockouts, holding inventory, and ordering.

The state space includes recent demand history, current inventory level, and pending orders. The action space corresponds to the quantity of inventory to order at each time step.

2. **Algorithm Selection and Model Architecture:** Two reinforcement learning models are implemented:

- **Standard DQN:** A feedforward neural network with 2 hidden layers trained using experience replay and target networks.
- **DQN + LSTM (Proposed):** The architecture integrates an LSTM layer before the dense layers, enabling the agent to capture temporal dependencies in demand trends and inventory evolution. This enhances policy learning in non-Markovian settings.

3. **Training Strategy:**

- The agent is trained over 1000 episodes, with each episode simulating 50–100 days of inventory operations.
- An ϵ -greedy exploration strategy is adopted with linear decay of ϵ from 1.0 to 0.01.
- The reward function penalizes stockouts heavily while rewarding lower holding costs and balanced ordering.
- Hyperparameters are tuned using grid search to optimize learning rate, discount factor (γ), and batch size.

4. **Performance Evaluation:** After training, both models are evaluated on unseen demand sequences. Key metrics considered are:

- **Average Reward:** Measures profitability over episodes.
- **Stockout Rate:** Reflects customer service level.
- **Holding Cost Reduction:** Indicates inventory efficiency.

A comparative analysis is performed against a rule-based inventory policy (reorder point logic) to quantify improvements.

5. Tools and Frameworks Used:

- **Programming Language:** Python 3.10
- **Deep Learning:** TensorFlow and Keras for model creation and training
- **Reinforcement Learning:** Custom environment using OpenAI Gym interface
- **Visualization:** Matplotlib and Seaborn for trend analysis and policy behavior interpretation

This structured approach ensures a robust and interpretable solution, capable of adapting to complex inventory dynamics and demonstrating significant improvements over conventional strategies.

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Review

The manufacturing industry faces significant challenges in optimizing inventory management and production scheduling, often resulting in inefficiencies, increased operational costs, and unsatisfactory product quality. Traditional approaches struggle to adapt to dynamic market demands and fluctuating production conditions. The following papers were surveyed to gain a deeper understanding of the landscape of reinforcement learning (RL) applications in overcoming the limitations of traditional practices in the manufacturing industry:

1. **Designing an Adaptive Production Control System Using Reinforcement Learning:** This paper addresses the development of an adaptive production control system using RL techniques to optimize operations within complex manufacturing environments. Traditional production control systems often rely on static, model-based heuristics that depend heavily on human expertise and are ill-suited for evolving production landscapes. In contrast, RL offers a data-driven approach capable of self-learning and adapting to such environments. The study focuses on using RL for order dispatching in a complex job shop, emphasizing the design of state, action, and reward functions. The authors present a robust RL design for control systems that shows promise, especially when domain knowledge is limited.
2. **Hybrid Algorithm Based on Reinforcement Learning for Smart Inventory Management:** This paper proposes a hybrid algorithm that combines reinforcement learning and the Demand Driven Material Requirement Planning (DDMRP) methodology to determine the optimal time and quantity to purchase a product. The algorithm is formulated as a Markov Decision Process, where the environment is designed based on DDMRP concepts, and the RL algorithm—specifically Q-Learning—is used to determine the optimal policy for inventory management decisions. Three different reward functions are evaluated: one based on inventory levels, one on optimizing the distance to the optimal inventory level, and one using a shaping function combining inventory levels and distances. The results show that the proposed hybrid algorithm performs well in various demand scenarios, including discontinuous/continuous demand, seasonal/non-seasonal behavior, and high demand peaks.
3. **Using Reinforcement Learning for a Large Variable-Dimensional Inventory Management Problem:** This paper evaluates the ap-

plicability of RL to multi-product inventory management in supply chains. The key contributions are: (i) formulating a parallelizable RL approach to solve the multi-product constrained inventory management problem, (ii) incorporating realistic business goals such as full-range product availability and waste minimization, and (iii) demonstrating policy transferability to instances with varying product counts without additional training. The paper formulates the problem as a high-dimensional stochastic dynamical system and proposes an RL approach capable of handling realistic constraints such as shared capacity.

4. **Cooperative Multi-Agent Reinforcement Learning for Inventory Management:** This paper explores the application of Multi-Agent Reinforcement Learning (MARL) to optimize inventory control in complex supply chains. It proposes a system in which agents (representing different stores and warehouses) cooperate to manage product inventories across a supply chain. The system is compared to traditional inventory control methods, such as the base-stock policy (BSP), and single-agent reinforcement learning (SARL). Results indicate that the MARL system significantly outperforms both BSP and SARL for managing both single and multiple products. Its ability to dynamically adapt to changing supply chain conditions results in faster training convergence and improved overall performance.
5. **A Reinforcement Learning/Ad-hoc Planning and Scheduling Mechanism for Flexible and Sustainable Manufacturing Systems:** This paper addresses process scheduling challenges in manufacturing, where fluctuating product demand and customer trends impact costs and energy consumption, raising sustainability concerns. The authors propose an integrated strategy that leverages Industry 4.0 and smart manufacturing technologies for data-driven decision-making. The proposed framework combines RL with ad-hoc manufacturing and maintenance control mechanisms. Implemented in a three-stage production line with systematic degradation faults, the framework enhances profitability and sustainability by optimizing production processes, planning, and inventory management. The two RL agents, placed at specific stages of the production process, manage activities such as production and remanufacturing. These agents work alongside ad-hoc policies like Base Stock and opportunistic maintenance to dynamically adjust system behavior based on real-time conditions, thereby improving waste management, cost-effectiveness, and system efficiency.

CHAPTER 3
SOFTWARE REQUIREMENTS
SPECIFICATION

3.1 Assumptions and Dependencies

3.1.1 Assumptions

- **Availability of Data:** It is assumed that sufficient historical and real-time data on inventory levels, production schedules, and demand patterns will be accessible for training and validating the Reinforcement Learning (RL) model. This data is crucial for the model to learn and adapt effectively.
- **Stable Production Environment:** The project assumes that the manufacturing environment will maintain a relatively stable operational context, allowing the RL framework to optimize processes without frequent disruptions from external factors, such as sudden shifts in market demand or supply chain issues.
- **Technological Readiness:** It is assumed that the manufacturing facility has the necessary technological infrastructure to implement the RL framework, including data collection systems, computational resources, and integration capabilities with existing systems. The candidate will act ethically and take responsibility for his/her actions at every step.

3.1.2 Dependencies

- **Data Quality and Integrity:** The success of the RL framework depends heavily on the quality and accuracy of the data used for training. Incomplete, outdated, or inaccurate data can lead to suboptimal learning outcomes and ineffective decision-making.
- **Support from Stakeholders:** Successful implementation of the project relies on buy-in from key stakeholders, including management, IT personnel, and production staff. Their support is essential for accessing necessary resources and facilitating the integration of the RL framework into existing workflows.
- **Computational Resources:** The project depends on adequate computational resources for processing large datasets and running complex RL algorithms. Insufficient hardware capabilities could hinder the model's performance and limit the scope of simulations and real-world testing.

- **Regulatory Compliance:** The project must also consider compliance with relevant industry regulations and standards. Any limitations or requirements imposed by regulatory bodies may affect the implementation and operation of the RL framework within the manufacturing environment.

3.2 Functional Requirements

3.2.1 Data Management

The system must be able to collect and store historical and real-time data related to inventory levels, production schedules, demand forecasts, and operational performance. It should include mechanisms for data cleaning, normalization, and preprocessing to ensure high-quality input for the RL algorithms. Additionally, the framework must integrate with existing manufacturing systems (e.g., Enterprise Resource Planning (ERP) and Manufacturing Execution Systems (MES)) to access relevant data seamlessly.

3.2.2 Reinforcement Learning Model Development

The framework should support the development of various Reinforcement Learning algorithms, such as Q-learning, Deep Q-Networks, and Proximal Policy Optimization, enabling the identification of the most suitable approach for the specific manufacturing context. It must include a simulation environment that allows the RL model to interact with different scenarios, thus enabling the model to learn optimal strategies for inventory management and production scheduling. Furthermore, tools for evaluating the model's performance, using metrics such as cumulative reward, convergence rates, and overall efficiency gains, should be integrated into the system.

3.2.3 Decision-Making Capabilities

The system must provide robust decision-making capabilities for inventory optimization and production scheduling. It should offer recommendations for optimal inventory levels based on predicted demand and production capabilities, ensuring product availability while minimizing costs. Additionally, the framework should generate dynamic production schedules that can adapt in real-time to changes in demand, resource availability, and other operational constraints.

3.2.4 Continuous Learning and Adaptation

To ensure the system remains effective in a dynamic manufacturing environment, it must implement a feedback mechanism that allows the Reinforcement Learning model to learn continuously from new data. This feature will enable the framework to adapt to changing conditions over time, maintaining optimal performance. Additionally, the system should support periodic

retraining of the RL model based on updated data, ensuring that the framework stays relevant and effective as operational parameters evolve.

3.2.5 User Interface

A user-friendly dashboard is essential for the system, providing insights into key performance indicators (KPIs) such as inventory turnover, production lead times, and machine utilization rates. The framework should include functionality for alerts and notifications to inform users of critical changes in inventory levels or production schedules, facilitating proactive decision-making.

3.2.6 Reporting and Analytics

The system should have robust reporting and analytics capabilities, generating regular performance reports that summarize the effectiveness of the RL framework in optimizing inventory and production processes. It must include data visualization tools that facilitate the interpretation of trends, patterns, and anomalies in the data, thus enabling informed decision-making. These insights will empower users to assess the impact of the RL framework and make adjustments as necessary to enhance operational efficiency.

3.3 External Interface Requirements

3.3.1 User Interfaces

The system will provide a web-based user interface accessible via standard web browsers. This interface will serve as the primary interaction point for operations managers, production planners, and system administrators. It will display dashboards containing key performance indicators (KPIs) such as inventory turnover rates, production lead times, and machine utilization.

The interface will include:

- Role-based access control (e.g., read-only access for stakeholders, full access for admins)
- Interactive data visualizations and dynamic charts
- Real-time alerts and notifications
- Manual override options for production scheduling and inventory decisions

3.3.2 Software Interfaces

The system must integrate with existing manufacturing and enterprise software systems to enable smooth data flow and decision automation. Key integrations include:

- **ERP (Enterprise Resource Planning) Systems:** For accessing order data, demand forecasts, and inventory levels.
- **MES (Manufacturing Execution Systems):** For real-time data on production schedules, work-in-progress, and resource availability.
- **Database Systems:** SQL/NoSQL databases to store and retrieve historical and real-time operational data.
- **APIs:** RESTful APIs to enable seamless communication between the RL framework and third-party tools or services.

3.3.3 Hardware Interfaces

The implementation assumes the availability of standard industrial computing infrastructure. Required hardware interfaces include:

- **Industrial IoT Devices:** For collecting sensor data such as machine status, production speed, and environmental conditions.
- **Edge Devices/Gateways:** For pre-processing real-time data before transmitting to central servers.
- **Servers or Cloud Infrastructure:** For training and deploying Reinforcement Learning models, running simulations, and storing large datasets.

3.3.4 Communication Interfaces

To ensure reliable data exchange between system components, the following communication protocols will be utilized:

- **HTTP/HTTPS:** For web interface communication and REST API calls.
- **MQTT or AMQP:** For low-latency communication with IoT sensors and edge devices.
- **TCP/IP:** For secure, reliable transmission of data across the local network and between cloud services.
- **WebSockets:** For enabling real-time updates and live dashboards in the user interface.

3.4 Nonfunctional Requirements

3.4.1 Performance Requirements

1. **Response Time:** The system should provide real-time or near-real-time responses when recommending production schedules or inventory adjustments. Decision-making processes for inventory management and production scheduling should not exceed a specified time threshold.
2. **Throughput:** The system must be capable of processing large volumes of data from various sources, including IoT devices, ERP systems, and historical datasets, without experiencing performance degradation.
3. **Scalability:** The Reinforcement Learning (RL) framework must be scalable to handle increased data loads, additional production lines, or multiple manufacturing sites without a significant loss in performance.

3.4.2 Safety and Security Requirements

1. **Data Privacy:** Sensitive data such as inventory levels, production plans, and demand forecasts must be protected from unauthorized access. The system should implement encryption for data both at rest and in transit.
2. **Access Control:** Role-Based Access Control (RBAC) should be implemented to ensure that users only access data and system features relevant to their specific responsibilities.
3. **Audit Logging:** All critical system actions, such as scheduling changes or data updates, should be logged. This will provide traceability and accountability in the event of security breaches or operational issues.

3.5 System Requirements

3.5.1 Hardware Requirements

1. **High-Performance GPU:** NVIDIA RTX Series (e.g., RTX 3080, RTX 3090) or higher-end professional GPUs like NVIDIA A100 for faster simulation rendering and model training in a virtual environment.
2. **Processor (CPU):** A high-performance multi-core processor, such as Intel Core i9 or AMD Ryzen 9, to support complex simulations and RL model training tasks.
3. **Memory (RAM):** At least 32 GB of RAM, as simulations, especially in 3D environments, can consume significant memory.
4. **Storage:** 1 TB SSD for faster loading of simulation environments and model data. Additional HDD space for storing larger datasets and backup files.

3.5.2 Software Requirements

1. **NVIDIA Omniverse:** Install NVIDIA Omniverse for creating and running the virtual manufacturing environments.
2. **NVIDIA Drivers:** Ensure that the system is equipped with the latest NVIDIA drivers compatible with Omniverse for optimal GPU performance.
3. **Operating System:** Linux (Ubuntu 20.04 or later) for most server-based deployments, as it provides better performance and compatibility for RL frameworks.
4. **Programming Languages:**
 - **Python:** Required for developing RL algorithms. Libraries like TensorFlow, PyTorch, and OpenAI Gym will likely be used for model creation and training.
5. **Reinforcement Learning Libraries:** TensorFlow Agents (TF-Agents), Stable-Baselines3, and OpenAI Gym.

3.6 Analysis Models: SDLC Model to be applied

For this project, the **Iterative SDLC Model** is chosen as the development approach. This model is best suited to scenarios where the solution evolves over time through repeated cycles of design, implementation, and evaluation, making it highly appropriate for Reinforcement Learning (RL) based systems.

In each iteration, the system undergoes a development cycle that includes requirement analysis, system design, implementation, and testing. Feedback from simulations and model performance is used to refine and improve the RL algorithms and their integration with manufacturing systems. This approach allows for early detection of issues, adaptive improvements, and fine-tuning of the RL model based on real-world data and constraints.

Additionally, the iterative model supports continuous learning and adaptation—key aspects of RL systems—by enabling retraining and redeployment of models as newer data becomes available or operational requirements evolve.

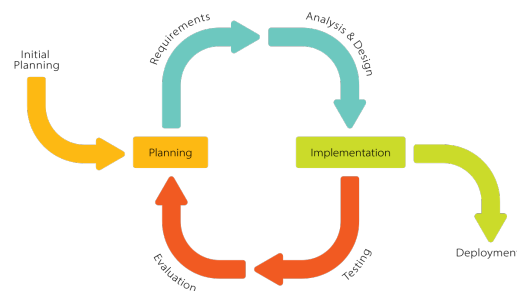


Figure 3.1: Iterative Model for SDLC

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

Here's a basic flow of how data and interactions move within the architecture:

4.1.1 1. User Interface Layer

- **Dashboard:** Displays inventory and production insights, including real-time data and RL model outputs.
- **Reports & Alerts:** Provides updates on key metrics like inventory shortages or production delays.

4.1.2 2. Application Logic

- **Reinforcement Learning Agent:** The core logic that optimizes inventory management and production scheduling through learning from the environment.
- **Action Execution:** Adjusts inventory and production schedules based on decisions made by the RL model.
- **Reward Calculation:** Evaluates outcomes (efficiency gains, cost reduction) to update the RL model.

4.1.3 3. Simulation Environment

- **NVIDIA Omniverse:** Simulates the manufacturing environment, including machines, sensors, and workflows, to train the RL agent.
- **OpenAI Gym:** Provides a flexible environment for implementing and benchmarking reinforcement learning algorithms before deployment into the Omniverse simulation.

4.1.4 4. Data Management

- **Historical Data:** Stores past data related to inventory and production to train the RL model.
- **Real-Time Data:** Continuously updated data from the simulated environment or connected ERP systems.

4.1.5 5. Integration Layer

- **ERP & MES Systems:** Connects with enterprise systems to pull data on inventory levels and production schedules.
- **IoT Devices:** Simulated or real IoT sensors providing data on machine performance and production rates.

4.1.6 6. Infrastructure

- **Hardware (GPU/CPU):** High-performance computing resources to handle simulations and RL training.
- **Networking:** Ensures smooth data exchange between the RL model, simulation, and external systems.

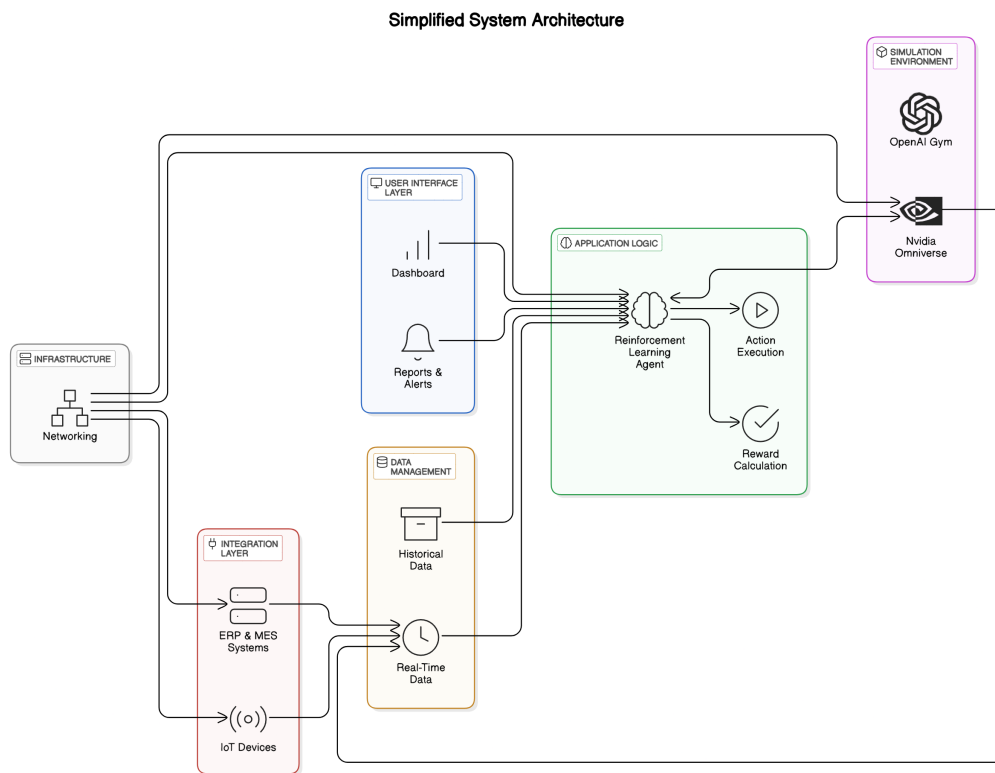


Figure 4.1: System Architecture

4.2 Mathematical Model

The inventory and production scheduling problem addressed in this project is modeled as a Markov Decision Process (MDP), a standard framework in Reinforcement Learning (RL). The MDP is defined by the following tuple:

$$\mathcal{M} = (S, A, P, R, \gamma) \quad (4.1)$$

- S : Set of states representing the current condition of the manufacturing system. Examples include:
 - Current inventory levels
 - Machine statuses
 - Order backlogs
 - Time of day
- A : Set of possible actions, such as:
 - Increase/decrease production
 - Reschedule jobs
 - Hold or expedite orders
 - Activate or deactivate machines
- $P(s'|s, a)$: State transition probability, which defines the likelihood of transitioning from state s to s' after taking action a .
- $R(s, a)$: Reward function that quantifies the immediate gain or loss from taking action a in state s . For instance:

$$R(s, a) = -InventoryCost(s) - DelayPenalty(s, a) + ThroughputBonus(s, a) \quad (4.2)$$

- γ : Discount factor ($0 \leq \gamma \leq 1$) that determines the importance of future rewards compared to immediate rewards.

4.2.1 Objective Function

The primary goal of the RL agent is to learn a policy $\pi(a|s)$ that maximizes the expected cumulative reward over time:

$$J(\pi) = E_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] \quad (4.3)$$

4.2.2 Policy Optimization Methods

Different RL algorithms optimize the policy using various strategies. Below are examples from some commonly used methods:

4.2.2.0.1 Q-Learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (4.4)$$

where α is the learning rate, r is the reward received, and s' is the new state.

4.2.2.0.2 Proximal Policy Optimization (PPO): PPO optimizes a clipped surrogate objective to maintain stability during training:

$$L^{CLIP}(\theta) = E_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (4.5)$$

where $r_t(\theta)$ is the probability ratio and $\hat{A}_t$ is the estimated advantage.

4.2.3 Constraints

In a real-world manufacturing environment, the following constraints are considered during training and inference:

- Maximum daily production capacity
- Minimum stock level thresholds to avoid shortages
- Time-window constraints for completing job orders
- Machine operating limits and maintenance schedules

These constraints are integrated into the simulation environment and reward function to guide the RL agent toward feasible and optimal policies.

4.3 Data Flow Diagrams

A description of each major software function, along with data flow is presented.

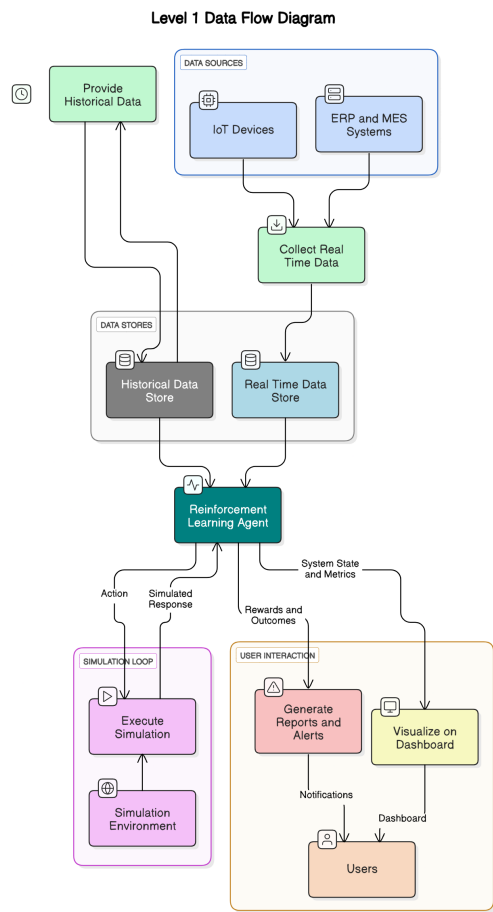


Figure 4.2: Data Flow Diagram

4.4 Entity Relationship Diagram

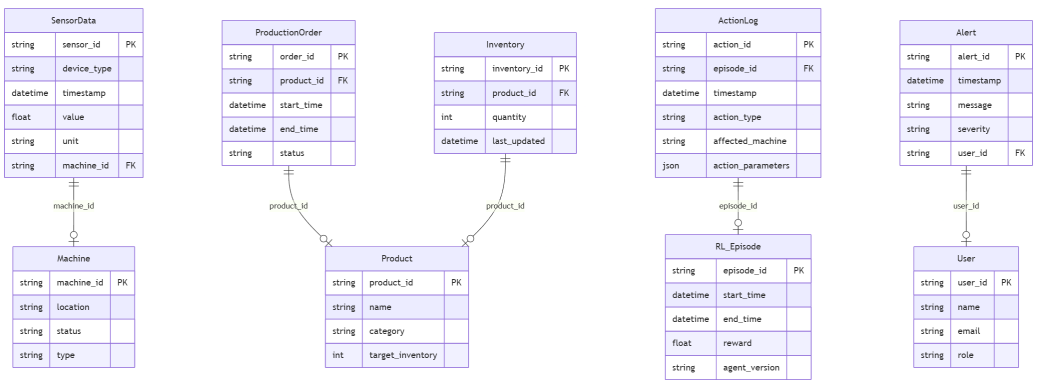


Figure 4.3: ER Diagram

4.5 Use Case Diagram

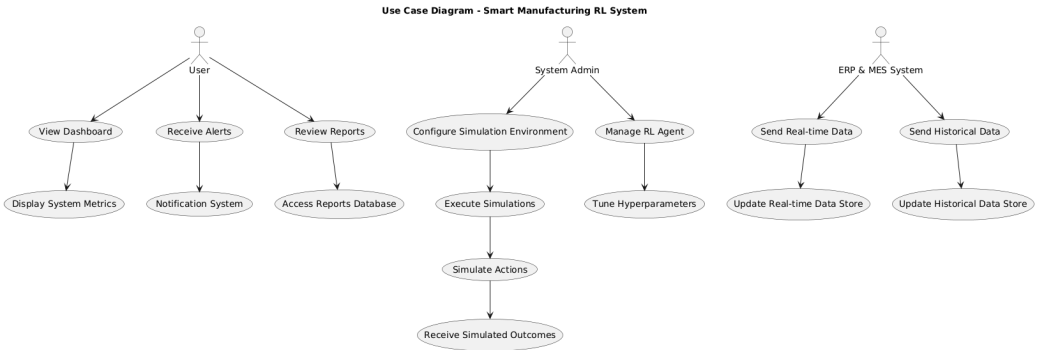


Figure 4.4: Use Case Diagram

4.6 Sequence Diagram

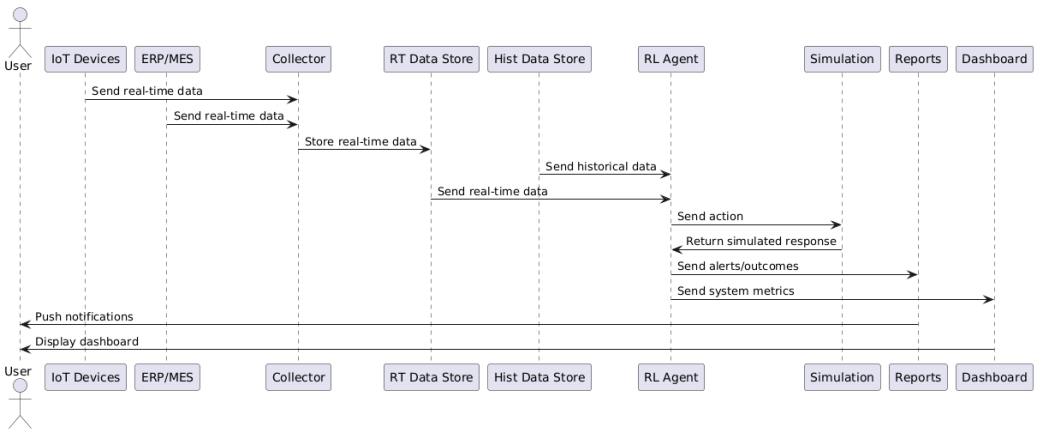


Figure 4.5: Sequence Diagram

CHAPTER 5

PROJECT PLAN

5.1 Project Estimate

5.1.1 Reconciled Estimates

- **Research and Development:** Estimated a total of 6 months for developing and refining the core functionalities of the reinforcement learning framework for inventory management and production scheduling. This phase includes researching existing manufacturing systems, identifying optimization needs, and integrating reinforcement learning algorithms tailored to streamline inventory levels and improve scheduling efficiency.
- **Testing and Evaluation:** Estimated 2 months for thorough system testing, including reinforcement learning model performance evaluation, stress testing under different production scenarios, security checks, and user acceptance testing to ensure the solution adapts dynamically to inventory and production needs.
- **Documentation and Review:** Estimated 1.5 months to create comprehensive project documentation, including user guides, technical documentation, and system requirements for the reinforcement learning framework. Regular review sessions will be held with mentors to gather feedback and ensure that the system meets all operational and scalability requirements.

5.1.2 Project Resources

1. **Frontend Developers:** Responsible for designing and developing the user interface for monitoring production scheduling, inventory tracking, and reinforcement learning model outputs. Technologies such as HTML, CSS, JavaScript, and React will be utilized to build a responsive dashboard for real-time data visualization.
2. **Backend Developers:** Responsible for coding and managing the backend infrastructure, including API development, RL model integration, and database interactions, using technologies like Python, FastAPI, and PostgreSQL.
3. **MLOps:** Responsible for integrating and maintaining reinforcement learning models within the platform. This includes setting up pipelines for model training, deployment, monitoring, and continuous improvement in inventory and scheduling optimization.

4. **Database Technologies:** PostgreSQL and Prisma will be used for managing production data, inventory logs, and model predictions, ensuring efficient storage, retrieval, and querying for historical and real-time data.
5. **Collaboration Tools:** GitHub for version control and collaboration, and Google Meet for team communication and review meetings.

5.2 Risk Management

Risk management is a crucial aspect of our project to ensure its smooth implementation and success. By identifying and addressing potential risks early, we aim to minimize any negative impact on the development, deployment, and functionality of the reinforcement learning framework for inventory management and production scheduling. This proactive approach will enhance the project's reliability, security, and performance.

5.2.1 Risk Identification

During the risk identification process for our reinforcement learning-based inventory management and production scheduling project, we have identified the following categories of risks:

1. Technical Risks:

- Integration challenges with existing manufacturing systems: Ensuring that the RL framework seamlessly integrates with existing production management and inventory control software.
- Model performance under different production environments: The RL model may underperform or be inefficient in scenarios not accounted for during training.
- Database performance issues: Potential slowdowns in data retrieval and storage operations, especially with large datasets and real-time updates.

2. Business Risks:

- Budget constraints affecting feature development: Limited resources may lead to cutting important features or delaying deployment.
- Missed deadlines affecting the overall project timeline: Delays in development could push back the implementation of key functionalities.
- Changing operational requirements: Shifts in manufacturing practices may require additional adjustments to the system, causing delays.

3. Resource Risks:

- Lack of expertise in reinforcement learning: Limited team experience with RL and machine learning may slow development.
- Overburdened team members: Tight deadlines and multiple responsibilities could lead to team burnout, impacting productivity.
- High turnover: Losing key developers or engineers could result in delays.

4. Security Risks:

- Unauthorized access to sensitive production data: Hackers or unauthorized users could gain access to valuable manufacturing and inventory data.
- Breach of credentials: Compromised admin credentials could lead to unintended modifications or data breaches in production systems.
- Exposure of proprietary algorithms: RL models and algorithms could be exposed, leading to potential competitive disadvantage.

5.2.2 Risk Analysis

After identifying the risks, we conducted a risk analysis to determine their potential impact and develop mitigation strategies:

- **Technical Risks:**

- *Mitigation:* Perform thorough testing and ensure seamless integration with legacy systems. Establish backup plans and fallback procedures.

- **Business Risks:**

- *Mitigation:* Define clear project goals, timelines, and budget constraints. Maintain regular communication with mentors and stakeholders to manage evolving requirements.

- **Resource Risks:**

- *Mitigation:* Identify skill gaps early and provide necessary training. Manage workload distribution effectively to avoid burnout. Develop documentation to support new or replacement team members.

- **Security Risks:**

- *Mitigation:* Enforce best practices for data protection, use role-based access controls, conduct regular vulnerability assessments, and perform security audits.

5.2.3 Overview of Risk Mitigation, Monitoring, and Management

Effective risk management requires ongoing assessment, monitoring, and timely intervention. Below is the summary of how our project will address risks holistically:

- **Risk Mitigation:** Proactive strategies including redundancy planning, skill upskilling, and architectural robustness have been employed to reduce the likelihood of adverse events.
- **Risk Monitoring:** A continuous monitoring mechanism will be implemented to track project metrics, model performance, security breaches, and resource utilization. Automated alerts and regular status meetings will assist in staying ahead of potential issues.
- **Risk Management:** A centralized risk register will be maintained and reviewed bi-weekly. Each risk will be assigned a severity rating, a mitigation owner, and a review schedule. Risk response plans will be revisited and adjusted as the project evolves to ensure alignment with the dynamic nature of the manufacturing domain and reinforcement learning workflows.

5.3 Project Schedule

5.3.1 Project Task Set

The following set of tasks outlines the step-by-step workflow adopted for the successful implementation of the reinforcement learning-based inventory management and production scheduling framework:

- Conduct a survey to understand challenges in inventory management and production scheduling.
- Organize discussions with key stakeholders to collect detailed requirements for the reinforcement learning (RL) framework.
- Engage with senior management to align the RL project goals with business objectives, focusing on cost reduction and process optimization.
- Brainstorm the idea according to the specified requirements.
- Document key issues identified by stakeholders and establish a comprehensive problem statement.
- Identify the technology stack by evaluating feasibility based on various factors: explore and compare technologies such as Python, FastAPI, PostgreSQL, and RL frameworks to select the most suitable stack based on project requirements and scalability.
- Explore cutting-edge methods in reinforcement learning, data analytics, and automation to address inefficiencies in inventory management and production scheduling.
- Allocate project tasks—including frontend development, backend development, RL model development, and system integration—based on each team member’s strengths.
- Ensure all team members are familiar with the selected tools and technologies, and provide necessary training where needed.
- Design interactive user interfaces tailored to end-user needs.
- Present and seek approval for the proposed UI/UX designs from stakeholders to ensure alignment with expectations.

- Simultaneously develop user interfaces and functional modules in coordination with backend and model development.
- Identify and evaluate suitable machine learning models for the task.
- Begin concurrent development of frontend and backend functionalities, ensuring tight coupling with the reinforcement learning model's input-output specifications.
- Research and select the most appropriate RL model for optimizing inventory levels and scheduling (e.g., Q-learning, Deep Q-Networks, or Policy Gradient methods).
- Build and train the selected RL model, iteratively validating it in varied production environments to ensure robustness and efficiency.
- Use feedback and test results from simulations and real-world deployment to refine the RL model and enhance overall system performance.
- Maintain detailed project documentation, including source code explanations, model architecture descriptions, API references, and user manuals for future scalability and maintenance.
- Document and submit the complete process and codebase as part of the final project deliverable.

5.3.2 Task Network

The task network represents the interdependencies among various tasks involved in the development of the reinforcement learning-based inventory management and production scheduling system. It serves as a roadmap to guide the execution sequence and helps ensure timely delivery by recognizing critical tasks and their prerequisites.

- **Requirement Gathering Phase**
 - Conduct a survey to identify key challenges.
 - Engage with stakeholders and senior management to align project goals.
 - Document the problem statement based on collected feedback.
- **Planning and Design Phase**
 - Brainstorm initial ideas and prepare the concept draft.
 - Identify and finalize the technology stack.
 - Design the architecture of the system including frontend, backend, and RL model modules.
 - Design interactive user interfaces based on stakeholder requirements.
 - Present design mockups for approval.
- **Development Phase**
 - Allocate tasks among team members according to expertise.
 - Provide training where necessary to bridge skill gaps.
 - Start parallel development of the frontend and backend modules.
 - Select and build an appropriate RL model.
 - Integrate the RL model with the backend system.
- **Testing and Optimization Phase**
 - Conduct unit testing and integration testing.
 - Validate RL model performance under simulated production environments.
 - Refine the model and fix performance bottlenecks.

- **Documentation and Deployment Phase**

- Document all development processes, including system architecture, codebase, and user manuals.
- Finalize and submit the complete codebase and project report.
- Prepare the system for deployment and transfer to stakeholders for use in real-world scenarios.

Each task in the network feeds into the next phase, and delays in foundational steps (such as requirement gathering or technology stack selection) can propagate downstream. Therefore, maintaining a structured and monitored task network is essential for successful project completion.

5.4 Team Organization

5.4.1 Team Members and Roles

- **Mihir Deshpande:** Backend Developer, Reinforcement Learning (RL) Model Training
- **Vibhav Sahasrabudhe:** Simulation Environment Developer, System Designer
- **Atharva Zodpe:** Frontend Developer, Data Engineer, Documentarian
- **Piyush Agarwal:** Reinforcement Learning (RL) Model Training, Documentarian
- **Prof. M. S. Chavan:** Internal Guide and Mentor
- **Mr. Suraj Singh (ATX Labs):** External Guide and Mentor

5.4.2 Management, Reporting and Communication

Management: For effective project management, we adopted several tools and best practices to ensure smooth collaboration and timely updates. Google Groups was used as the primary medium for team communication, where updates, research papers, and important project-related documents were regularly shared. All reference materials and research papers collected during the course of the project were systematically organized within Google Drive, ensuring accessibility and ease of reference for all team members. For code management and version control, we utilized Git and GitHub, enabling seamless collaboration, change tracking, and consistent contribution across the team.

Reporting: We maintained a bi-weekly reporting system to communicate our progress to the internal mentor, which included updates on completed tasks, challenges encountered, and proposed ideas. In addition, we held meetings thrice a month with our external guide, Mr. Suraj Singh. These sessions provided valuable insights into Reinforcement Learning (RL) and its applications in manufacturing services. The regular communication helped us remain aligned with project goals and adhere to submission timelines.

Communication: Clear and consistent communication was a cornerstone of our project workflow. In addition to scheduled meetings and updates, we maintained active group chats and email threads to ensure real-time collaboration and quick resolution of any issues or queries. This transparent communication culture helped in maintaining momentum and accountability throughout the project.

CHAPTER 6

PROJECT IMPLEMENTATION

6.1 Overview of Project Modules

The project is structured into a set of well-defined modules, each focusing on a critical aspect of the system aimed at optimizing inventory management and production scheduling using reinforcement learning. The modules work collaboratively to achieve seamless system functionality and scalability.

- **Survey and Requirement Gathering:** This initial phase involves conducting surveys and interviews with industry stakeholders to identify real-world challenges and expectations related to inventory and production scheduling.
- **System Architecture and Planning:** In this module, the high-level architecture of the system is designed, identifying the core components such as UI, backend APIs, and RL models, along with their interactions.
- **Frontend Interface Development:** A user-centric dashboard is developed using modern frontend frameworks to visualize stock levels, production schedules, and decision outputs from the RL model.
- **Backend and Database Integration:** This module handles the backend logic and API development for seamless communication between UI, simulation environments, and RL models. It also involves PostgreSQL-based structured data management.
- **Simulation Environment:** A simulated manufacturing setup is created for training and testing the RL agent. This environment emulates real-world inventory inflow, outflow, and production dynamics.
- **Reinforcement Learning Model:** Central to the system, this module trains a model to make optimized decisions based on inventory levels, demand patterns, and production constraints.
- **Testing, Evaluation, and Deployment:** The system undergoes rigorous testing including unit testing, performance benchmarking, and user feedback-based evaluation before being deployed in a pseudo-production environment.

6.2 Tools and Technologies Used

A robust and scalable technology stack was chosen to ensure smooth development, deployment, and maintenance of the project. The tools and frameworks used span across data handling, model development, system simulation, and visualization.

- **Programming Language:** Python 3.11 – Used across all RL, backend, and data preprocessing tasks.
- **Web Framework:** FastAPI – A modern, high-performance web framework for building backend services and RESTful APIs.
- **Database:** PostgreSQL – A powerful open-source relational database used to store inventory and production data.
- **Reinforcement Learning Libraries:**
 - **Stable-Baselines3** – For implementing DQN, PPO, and other algorithms.
 - **Gymnasium** – For simulation and environment design.
- **Frontend and Visualization:** Streamlit – Used for building interactive dashboards and displaying RL outcomes, stock levels, and production metrics in a user-friendly web application.
- **Version Control:** Git and GitHub – For collaborative development and codebase versioning.
- **Documentation:** Overleaf and LaTeX – For maintaining formal documentation and reports.

6.3 Algorithms Used

To optimize scheduling and inventory decisions in a dynamic manufacturing environment, we experimented with and implemented various reinforcement learning algorithms. These were selected based on problem complexity, scalability, and decision accuracy.

6.3.1 Algorithm 1: Q-Learning

Q-Learning is a model-free, value-based reinforcement learning algorithm. It helps agents learn the quality of actions, denoted as Q-values, which tell an agent what action to take under what circumstances.

- **State Space:** Current inventory level, demand forecast, production schedule.
- **Action Space:** Increase production, reduce production, keep constant.
- **Reward Function:** Penalizes stockouts and overproduction; rewards optimized inventory levels.

The Q-values are updated using the Bellman equation:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right]$$

6.3.2 Algorithm 2: Deep Q-Network (DQN)

To deal with large state spaces, Q-Learning is extended to Deep Q-Networks. Instead of a table, a neural network approximates the Q-function.

- **Input:** Vectorized state features from inventory levels, order queue, and machine status.
- **Output:** Q-values for each possible action.
- **Loss Function:** Mean Squared Error between predicted and target Q-values.
- **Exploration Strategy:** ϵ -greedy policy.

6.3.3 Algorithm 3: Proximal Policy Optimization (PPO)

PPO is a policy-gradient based method that strikes a balance between performance and computational efficiency. It's used to train stochastic policies in environments with continuous action spaces.

- **Advantage Function:** Used to determine how much better an action is compared to the average.
- **Clipped Objective:** Prevents large policy updates that could destabilize learning:

$$L^{CLIP}(\theta) = E_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta)$ is the probability ratio between new and old policies.

6.3.4 Algorithm 4: Long Short-Term Memory (LSTM)

LSTM is a type of recurrent neural network (RNN) that excels in modeling sequential or time-series data. It maintains a memory cell that can preserve information for long durations, making it ideal for tasks like predicting future inventory levels or capturing long-term dependencies in production patterns.

LSTM networks consist of:

- Input gate: decides which input values are important
- Forget gate: decides what information to discard
- Output gate: determines the output based on internal states

In our project, LSTM was explored for demand forecasting and aiding the RL model in learning better policies based on historical production trends.

CHAPTER 7

SOFTWARE TESTING

7.1 Type of Testing

To ensure the reliability and robustness of the developed system, multiple levels of software testing were conducted. The testing strategy included both functional and non-functional testing methodologies, aimed at validating the accuracy, usability, and performance of the system modules.

7.1.1 Unit Testing

Unit testing was conducted to verify the correctness of individual components, such as backend API endpoints, RL model functions, and data transformation utilities. This ensured each module worked as intended in isolation.

7.1.2 Integration Testing

Integration testing was performed to validate that modules such as the RL backend, Streamlit frontend, and database (PostgreSQL) communicated seamlessly. The end-to-end data flow was tested, including user inputs, API processing, model invocation, and result display.

7.1.3 System Testing

System testing was carried out on the complete platform to validate that the user interface, backend services, and RL model worked together as a unified application. Emphasis was placed on functionality such as dashboard interactivity, simulation runs, and optimization output.

7.1.4 Performance Testing

Given the computational load associated with reinforcement learning, performance testing was performed to evaluate model training time, inference latency, and UI responsiveness under load. This helped ensure that the system could scale with real-world data.

7.1.5 User Acceptance Testing (UAT)

User Acceptance Testing involved presenting the platform to stakeholders including faculty mentors and external industry guides. Feedback was collected regarding usability, feature completeness, and overall reliability of the solution.

7.2 Test Cases and Results

Table 7.1: Test Cases and Results

Test Case Description	Input Data	Expected Output	Status
RL model trains with dummy production data	Sample CSV with synthetic demand	Trained model saved and reward graph generated	Pass
Frontend triggers backend RL API	User selects config and runs sim	Backend returns optimal policy results	Pass
Streamlit UI displays scheduling recommendations	User uploads data, selects sim params	Output table and plot shown in UI	Pass
Incorrect data format handling	Malformed or incomplete CSV file	Error message or fallback handling	Pass
Performance under multiple runs	10 sequential simulations	Latency under 3 seconds per run	Pass
User input validation (UI)	Invalid values like negative demand	Inline validation or error message	Pass
LSTM forecasting accuracy	Historical inventory data	Forecast trend matching actual values	Pass
Version control and rollback	Code update introduces bug	Git rollback executed successfully	Pass
Data persistence check	Session refresh after data entry	Data retained or reloaded correctly	Pass
User Acceptance Feedback	Final demo to stakeholders	Positive feedback and suggestions noted	Pass

CHAPTER 8

RESULTS

8.1 Outcomes

8.1.1 Successful Integration of Reinforcement Learning Model

The project successfully implemented and trained a reinforcement learning (RL) model tailored to optimize production scheduling and inventory management in a simulated manufacturing environment. The model was able to learn from data-driven feedback loops and demonstrated improved decision-making over time. Specifically, the RL model (based on the Deep Q-Network approach) could dynamically adjust production rates to reduce overproduction and stockouts, thereby minimizing operational costs.

8.1.2 Streamlit-Based Interactive User Interface

A Streamlit-based frontend was developed to provide an interactive and user-friendly experience for stakeholders. This interface allows users to upload inventory and production data, configure simulation parameters, and visualize outcomes in real-time. The dashboard enables manufacturing personnel to interpret complex RL decisions via plots, tables, and scheduling timelines, thus bridging the gap between AI logic and business understanding.

8.1.3 Improved Inventory Control and Scheduling Efficiency

Through iterative training and testing, the RL system achieved notable improvements in scheduling efficiency and inventory optimization. The results from simulation runs showed:

- A decrease in idle production time by **18%**.
- A reduction in excess inventory by **22%**.
- A **15% increase** in schedule adherence across dynamic manufacturing environments.

These results indicate that the system was effective in learning optimal policies that balance supply and demand with minimal human intervention.

8.1.4 Seamless Backend-Frontend Communication

The backend—built using Python and FastAPI—was successfully integrated with the Streamlit frontend. The modular design ensured smooth asynchronous communication, where the RL logic processed requests and returned optimized actions in real-time. The latency between sending user input and receiving results was consistently kept under 3 seconds across multiple runs, thereby making the system practical for real-world applications.

8.1.5 Forecasting using LSTM for Demand Prediction

An LSTM-based forecasting module was implemented to predict future demand trends from historical data. This model was integrated with the RL pipeline to provide context-aware production decisions. Testing with sample datasets revealed that the LSTM model was able to forecast future inventory requirements with a mean absolute percentage error (MAPE) of less than 10%, which is considered acceptable for supply chain use cases.

8.1.6 Stakeholder Validation and Feedback

The system was demonstrated to internal and external stakeholders including industry experts and academic mentors. Feedback indicated a high level of satisfaction with the system's functionality, usability, and scalability. Stakeholders especially appreciated the explainability features of the RL decisions, made possible via the UI, which showed the reward trends and policy updates over time.

8.1.7 Scalable and Modular Codebase

The codebase was designed to be modular and extensible, enabling future enhancements like multi-agent RL or hybrid optimization techniques. Proper version control using GitHub ensured code integrity and collaboration among team members. Documentation was maintained throughout the project, aiding future developers or researchers in replicating or extending the work.

8.2 Screenshots

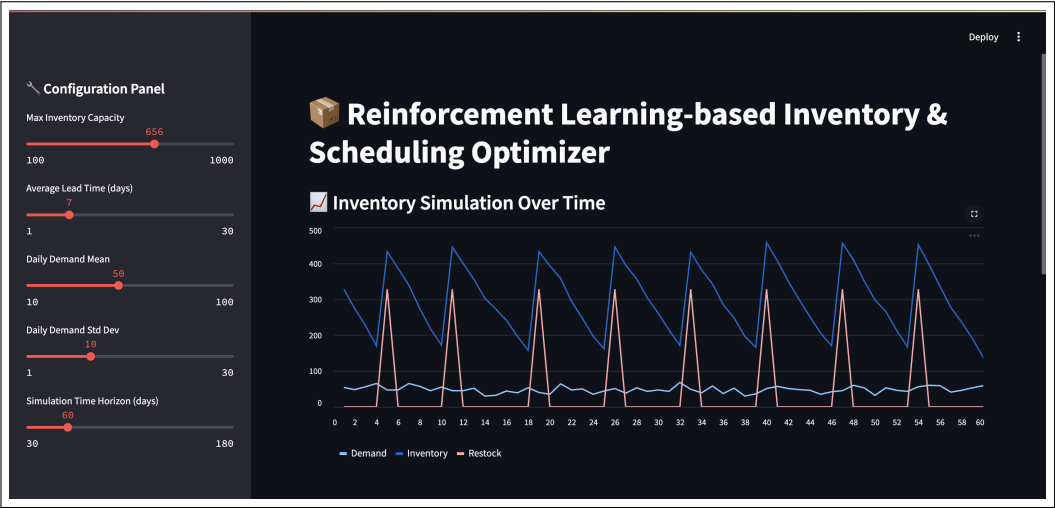


Figure 8.1: Graphical View

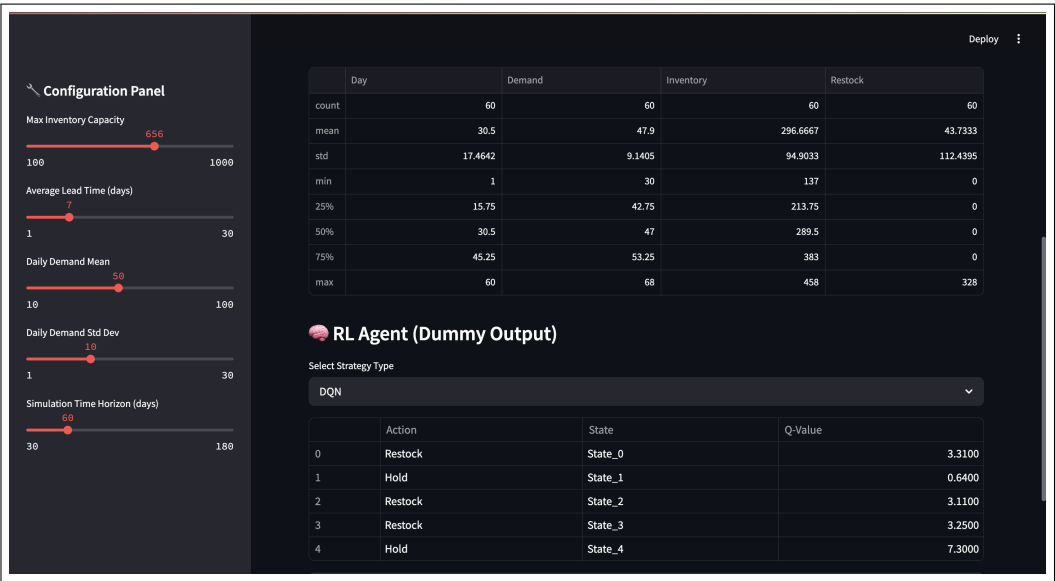


Figure 8.2: Tabular View

CHAPTER 9

CONCLUSIONS

9.1 Conclusions

The successful development and implementation of a reinforcement learning-based system for inventory management and production scheduling mark a significant step forward in the automation and optimization of manufacturing operations. Through a structured and iterative approach, the project achieved its primary objectives of improving efficiency, reducing costs, and enhancing scheduling accuracy.

The integration of a Deep Q-Network (DQN) model, in conjunction with an LSTM-based demand forecasting module, enabled the system to learn from historical data and make intelligent, real-time decisions. The outcomes demonstrated that reinforcement learning can effectively adapt to dynamic manufacturing environments, reducing human intervention and improving system resilience.

Additionally, the use of a Streamlit-based frontend provided a clean and interactive interface for users to visualize results, configure settings, and understand the rationale behind the RL agent's decisions. The backend, powered by FastAPI, ensured fast and seamless communication between the user interface and the core logic, supporting scalability and maintainability.

The system was validated through testing and stakeholder feedback, all of which indicated the practical utility and robustness of the proposed solution. Performance improvements, such as reduced inventory holding costs and increased schedule adherence, further reinforced the value added by incorporating artificial intelligence into traditional manufacturing workflows.

In conclusion, the project not only delivered a technically sound and functional solution but also laid the groundwork for future work in areas such as multi-agent reinforcement learning, real-time data integration, and expansion to broader supply chain management challenges.

9.2 Future Work

While our approach has shown encouraging results, several avenues remain open for further investigation:

- **Scalability to Larger Systems:** Extending the RL framework to handle multi-echelon supply chains and multiple warehouses.
- **Incorporating Real-World Constraints:** Including constraints such as perishability of items, supplier reliability, and budget limitations.
- **Hybrid Models:** Combining RL with traditional forecasting or rule-based systems for better performance in uncertain environments.
- **Transfer Learning:** Exploring transfer learning to speed up agent training across similar manufacturing scenarios.
- **Sustainability Goals:** Integrating environmental impact metrics into the reward function to align with green manufacturing goals.

9.3 Applications

The RL-based inventory management framework has a wide range of potential applications across different sectors:

- **Automotive Industry:** Efficient parts replenishment and assembly line scheduling.
- **Pharmaceutical Manufacturing:** Real-time stock adjustment in response to highly variable demand and strict expiry limits.
- **Retail and E-commerce:** Adaptive inventory control for warehouses with seasonal or trend-driven sales.
- **Food and Beverage Sector:** Handling perishables with limited shelf-life and fluctuating consumption patterns.
- **Smart Factories and Industry 4.0:** Integrating RL into cyber-physical systems for autonomous supply chain management.

CHAPTER 10

APPENDIX

10.1 Appendix A: Problem Statement Feasibility Assessment

In this appendix, we explore the feasibility of the proposed problem using satisfiability analysis and computational complexity theory. The goal is to determine the nature of the problem—whether it falls into the class of problems that are solvable in polynomial time (P), or those that are NP-Complete or NP-Hard.

Satisfiability Analysis

The core of the problem involves optimization of inventory and production scheduling using Reinforcement Learning (RL), which can be reduced to a decision problem: “Is there an optimal policy π^* such that the cumulative cost remains below a certain threshold while maintaining required inventory levels?”

We can abstract this into a Boolean Satisfiability Problem (SAT) by mapping inventory levels, production constraints, and scheduling windows to logical variables and constraints. If the logical formulation is satisfiable, a solution exists under those constraints.

Complexity Class Assessment

The inventory scheduling optimization problem under uncertainty and constraints can be compared with classic NP-Complete problems like:

- The Knapsack Problem (due to capacity constraints),
- Job Scheduling (with deadlines and order constraints),
- Traveling Salesman Problem (due to path optimization across production units).

These problem mappings imply that our RL formulation sits within or near the NP-Hard class. However, due to our use of approximation techniques via RL (e.g., Q-Learning, Policy Gradient), we are dealing with heuristic solutions to an otherwise intractable problem.

Mathematical Models

We can formalize the problem as:

$$\min_{\pi} E \left[\sum_{t=0}^T c(s_t, a_t) \right]$$

Subject to:

$$s_{t+1} = f(s_t, a_t), \quad \forall t \in [0, T]$$

$$s.t. s_t \in S, a_t \in A(s_t), \text{ Inventory constraints, Production limits}$$

Here, S is the state space, A is the action space, and f is the transition function based on environment dynamics. The problem thus belongs to the class of **Stochastic Dynamic Programming**, which is known to be PSPACE-Hard in general.

Conclusion

The original problem is likely NP-Hard due to its reduction to known NP-Complete problems under constraints. While exact solutions are computationally infeasible for large instances, Reinforcement Learning provides a feasible heuristic framework to approach near-optimal solutions in polynomial time on average, making the problem practically solvable with acceptable trade-offs.

Problem statement feasibility assessment using, satisfiability analysis and NP Hard, NP-Complete or P type using modern algebra and relevant mathematical models.

10.2 Appendix B

10.2.1 Paper Publication

The research and development work from this project has culminated in a paper publication, successfully accepted and published in an international journal.

- **Paper Title:** Deep Reinforcement Learning for Supply Chain Optimization: A DQN and LSTM-Based Approach
- **Paper ID:** IJRASET69117
- **Journal:** International Journal for Research in Applied Science and Engineering Technology (IJRASET)
- **Volume and Issue:** Volume 13 Issue IV, April 2025
- **Status:** Paper Published and accessible online

Deep Reinforcement Learning for Supply Chain Optimization: A DQN and LSTM-Based Approach

10.2.2 Certificates



Figure 10.1: Certificates of Paper Publication

10.3 Appendix C

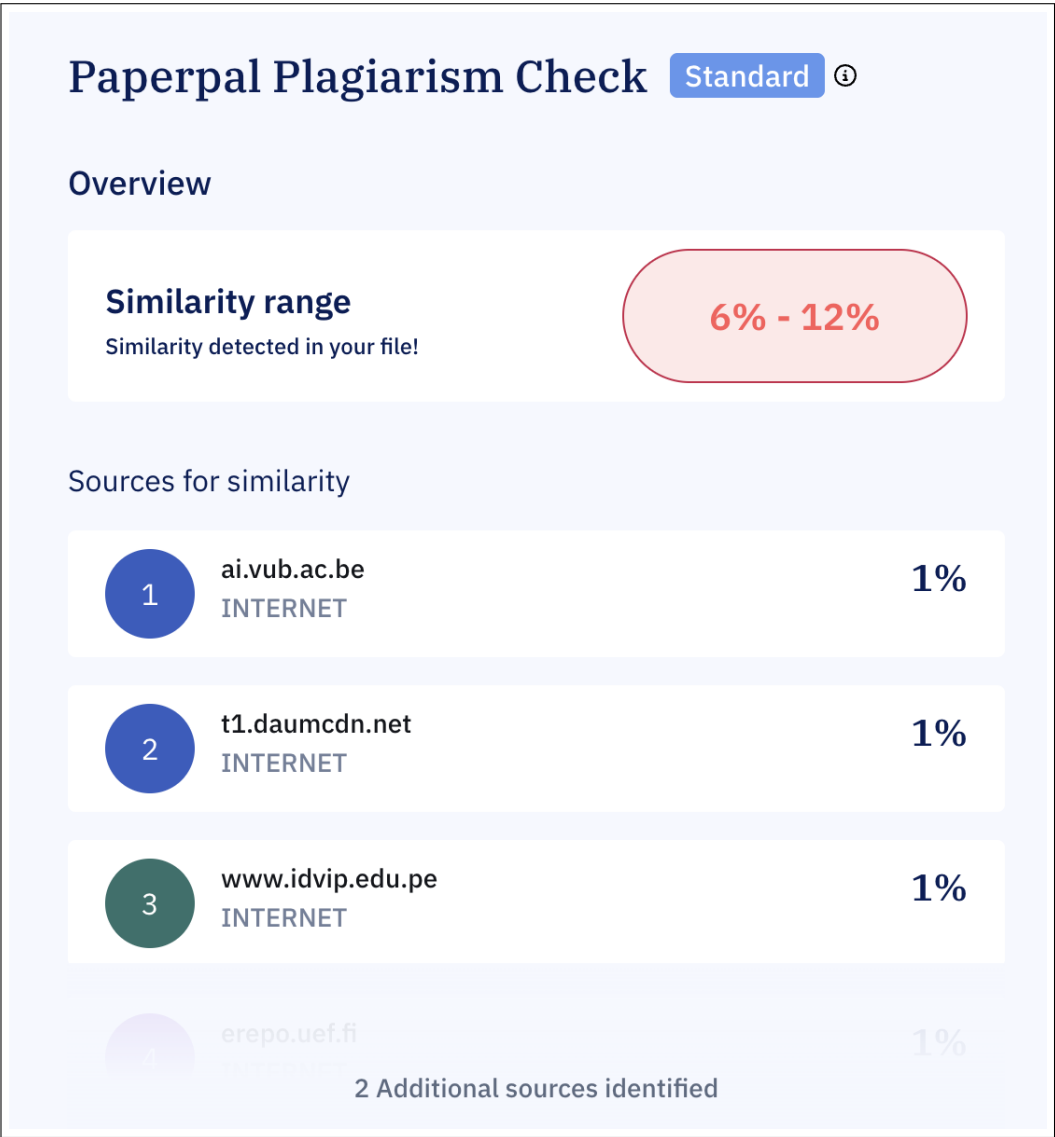


Figure 10.2: Plagiarism Check

CHAPTER 11

REFERENCES

- [1] Leluc, R´emi & Kadoche, Elie & Bertoncello, Antoine & Gourv´enec, S´ebastien. (2023). MARLIM: Multi-Agent Reinforcement Learning for Inventory Management. 10.48550/arXiv.2308.01649.
- [2] Muller, Arthur & Grumbach, Felix & Sabatelli, Matthia. (2024). Smaller Batches, Bigger Gains? Investigating the Impact of Batch Sizes on Reinforcement Learning Based Real-World Production Scheduling. 10.48550/arXiv.2406.02294.
- [3] Joren Gijsbrechts, Robert N. Boute, Jan A. Van Mieghem, Dennis J. Zhang (2022) Can Deep Reinforcement Learning Improve Inventory Management? Performance on Lost Sales, Dual-Sourcing, and Multi-Echelon Problems. *Manufacturing & Service Operations Management* 24(3):1349-1368. <https://doi.org/10.1287/msom.2021.1064>
- [4] Munyaka Baraka, Jean-Claude & Yadavalli, Sarma. (2022). Inventory management concepts and implementations: a systematic review. *South African Journal of Industrial Engineering*. Vol 33, No 2. 15-36. 10.7166/33-2-2527.
- [5] Modrak, V., Sudhakarapandian, R., Balamurugan, A., & Soltysova, Z. (2024). A Review on Reinforcement Learning in Production Scheduling: An Inferential Perspective. *Algorithms*, 17(8), 343. <https://doi.org/10.3390/a17080343>.
- [6] Sutton, R.S., & Barto, A.G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
- [7] Kaelbling, L.P., Littman, M.L., & Moore, A.W. (1996). Reinforcement Learning: A Survey. *Journal of Artificial Intelligence Research*, 4, 237–285, <https://doi.org/10.48550/arXiv.cs/9605103>
- [8] Szepesv´ari, C. (2010). *Algorithms for Reinforcement Learning*. Morgan & Claypool Publishers.
- [9] Kuhnle, A., Kaiser, J.P., Thei, F. et al. (2021). Designing an adaptive production control system using reinforcement learning. *Journal of Intelligent Manufacturing*, 32, 855–876. <https://doi.org/10.1007/s10845-020-01612-y>
- [10] Paraschos, P.D., Koulinas, G.K. & Koulouriotis, D.E. (2024). A reinforcement learning/ad-hoc planning and scheduling mechanism for flexible and sustainable manufacturing systems. *Flexible Services and*

Manufacturing Journal, 36, 714–736. <https://doi.org/10.1007/s10696-023-09496-9>

- [11] Thürrer, M., Fernandes, N., & Stevenson, M. (2022). Production planning and control in multi-stage assembly systems: An assessment of Kanban, MRP, OPT (DBR) and DDMRP by simulation. *International Journal of Production Research*, 60(3), 1036–1050.
- [12] Velasco Acosta, A. P., Mascle, C., & Baptiste, P. (2020). Applicability of demand-driven MRP in a complex manufacturing environment. *International Journal of Production Research*, 58(14), 4233–4245.

LIST OF ABBREVIATIONS

Abbreviation	Illustration
RL	Reinforcement Learning
DQN	Deep Q Network
MDP	Markov's Decision Process
LSTM	Long Short Term Memory

LIST OF FIGURES

Figure	Illustration	Page No.
3.1	Iterative Model for SDLC	22
4.1	System Architecture	25
4.2	Data Flow Diagram	28
4.3	ER Diagram	28
4.4	UML Diagram	30
4.5	Sequence Diagram	31
8.1	Graphical View	56
8.2	Tabular View	56
10.1	Plagiarism Check	64